

Introduction to Parallel Algorithms and Programming

Session 1: Introduction to parallel computing

Oguz Kaya

Assistant Professor

Université Paris-Saclay and LISN, Gif-sur-Yvette, France

Objectives

- Understand what **parallel computing** is, as opposed to **sequential computing**
- Understand **why** it is important, and unavoidable today
- Discover **applications** that require it
- Learn the vocabulary used throughout the course: **speedup**, **efficiency**, **Amdahl's law**, **data/task parallelism**, **SIMD/MIMD**
- Discover the **one principle** on which the whole course rests:
processing unit i executes the same program on index i
- Apply it for the first time today with **SIMD** programming

Outline

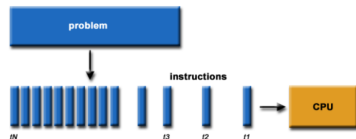
- 1 Introduction
- 2 Parallel computing, why?
- 3 Measuring parallel performance
- 4 Classification of parallel machines
- 5 Kinds of parallelism in programs
- 6 Parallel programming models
- 7 Course organisation

- 1 Introduction
- 2 Parallel computing, why?
- 3 Measuring parallel performance
- 4 Classification of parallel machines
- 5 Kinds of parallelism in programs
- 6 Parallel programming models
- 7 Course organisation

Sequential programs

Traditionally, programs are based on a **sequential** execution:

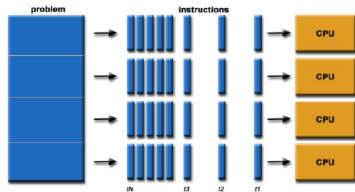
- A problem is split into a series of basic instructions.
- These instructions are executed **sequentially** one after the other.
- Instructions are executed by a **single processor**.
- At a given time unit (clock tick), **a single instruction** is executed.
- Runtime of the program (or performance) mostly depends on the **frequency (Hz)** of the processor.



Parallel programs

A **parallel program** uses multiple processing units to solve a problem:

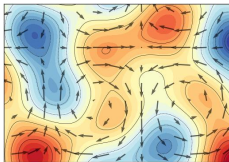
- A problem is split into subproblems that can be executed simultaneously.
- Each subproblem is split into basic instructions.
- Instructions of each subproblem are executed **in parallel**, one processing unit per subproblem.
- The performance is mostly determined by:
 - frequency (Hz)
 - number of processing units
 - degree of parallelism of the problem
 - other characteristics (load balancing, overheads due to parallelism)



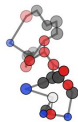
- 1 Introduction
- 2 Parallel computing, why?**
- 3 Measuring parallel performance
- 4 Classification of parallel machines
- 5 Kinds of parallelism in programs
- 6 Parallel programming models
- 7 Course organisation

Applications of parallel computing: science and engineering

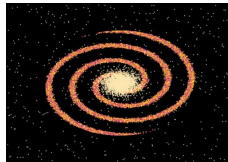
Numerous applications require immense compute power:



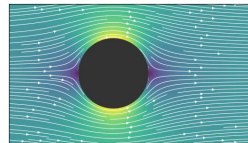
Weather and climate simulation



Molecular dynamics, drug design



Astrophysics, cosmology



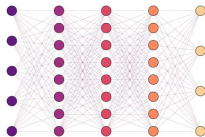
Fluid dynamics, aircraft and car design

Some orders of magnitude

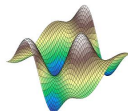
- A weather forecast is useless if computing it takes longer than the weather itself: the resolution of the model is dictated by the number of processors available.
- Simulating one microsecond of a protein's motion requires $\approx 10^{18}$ floating point operations: **3 years** on one core, **one day** on a thousand.

Applications of parallel computing: everyday computing

... and it is not only for scientists:



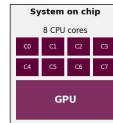
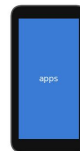
Training and serving AI models



Video games, 3D rendering



Video streaming and encoding



Your phone: 6–8 cores + GPU

Some orders of magnitude

- Training a large language model: $\approx 10^{25}$ floating point operations. One CPU core at 10^{10} op/s → **30 million years**; tens of thousands of GPUs in parallel → **a few months**.
- Rendering one frame of a modern game: $\approx 10^{10}$ operations, 60 times per second.
- Your browser, video decoder, camera and games are all parallel programs.

Maximum frequency of a CPU in 2002

What is the maximum frequency of a CPU in 2002?



Intel Pentium 4, Northwood

Maximum frequency of a CPU in 2002

What is the maximum frequency of a CPU in 2002?

- 3.06 GHz



Intel Pentium 4, Northwood

Maximum frequency of a CPU today

What is the maximum frequency of a CPU today?



Intel Core i9-10900K, 2020

Maximum frequency of a CPU today

What is the maximum frequency of a CPU today?

- 5.3 GHz in 2020, barely 6 GHz today: **less than $2\times$ in 20 years.**



Intel Core i9-10900K, 2020

Maximum frequency of a CPU today

What is the maximum frequency of a CPU today?

- 5.3 GHz in 2020, barely 6 GHz today: **less than $2\times$ in 20 years.**
- Then peak performance is also less than $2\times$?
 ≈ 6 Gops/s?



Intel Core i9-10900K, 2020

Maximum frequency of a CPU today

What is the maximum frequency of a CPU today?

- 5.3 GHz in 2020, barely 6 GHz today: **less than $2\times$ in 20 years.**
- Then peak performance is also less than $2\times$?
 ≈ 6 Gops/s?
- No! ≈ 1.7 Tflops/s (1700 Gflops/s) already in 2020. How?



Intel Core i9-10900K, 2020

Maximum frequency of a CPU today

What is the maximum frequency of a CPU today?

- 5.3 GHz in 2020, barely 6 GHz today: **less than $2\times$ in 20 years.**
- Then peak performance is also less than $2\times$? ≈ 6 Gops/s?
- No! ≈ 1.7 Tflops/s (1700 Gflops/s) already in 2020. How?
- Thanks to the **parallelism inside the CPU**: 10 cores, each having two 8-wide vector units:
 $10 \times 2 \times 8 \times 2 \times 5.3 \text{ GHz} \approx 1700 \text{ Gflops/s}.$



Intel Core i9-10900K, 2020

Maximum frequency of a CPU today

What is the maximum frequency of a CPU today?

- 5.3 GHz in 2020, barely 6 GHz today: **less than $2\times$ in 20 years.**
- Then peak performance is also less than $2\times$? ≈ 6 Gops/s?
- No! ≈ 1.7 Tflops/s (1700 Gflops/s) already in 2020. How?
- Thanks to the **parallelism inside the CPU**: 10 cores, each having two 8-wide vector units:
 $10 \times 2 \times 8 \times 2 \times 5.3 \text{ GHz} \approx 1700 \text{ Gflops/s}$.
- **A sequential program uses less than 1% of this.** The rest can only be obtained through parallel programming.



Intel Core i9-10900K, 2020

- 1 Introduction
- 2 Parallel computing, why?
- 3 Measuring parallel performance**
- 4 Classification of parallel machines
- 5 Kinds of parallelism in programs
- 6 Parallel programming models
- 7 Course organisation

Speedup and efficiency

Speedup (or acceleration): How much faster does a parallel program run?

Efficiency: How well are the processing units used?

- Sequential execution time: $T(1)$
- Parallel execution time using N processing units: $T(N)$
- **Speedup:** $S(N) = T(1)/T(N)$
- **Efficiency:** $E(N) = S(N)/N$
- Ideal case: $S(N) = N$, $E(N) = 1$. In practice, $S(N) \leq N$ (why?)



Cray-1 Supercomputer (1975) with peak performance of ≈ 160 Mflops/s

Speedup and efficiency: an example

A program takes $T(1) = 120$ s on one processing unit. We run it on $N = 2, 4, 8, 16$ units and measure:

N	$T(N)$	$S(N) = T(1)/T(N)$	$E(N) = S(N)/N$
1	120 s	1.0	100%
2	62 s		
4	34 s		
8	21 s		
16	16 s		

Speedup and efficiency: an example

A program takes $T(1) = 120$ s on one processing unit. We run it on $N = 2, 4, 8, 16$ units and measure:

N	$T(N)$	$S(N) = T(1)/T(N)$	$E(N) = S(N)/N$
1	120 s	1.0	100%
2	62 s	1.9	97%
4	34 s		
8	21 s		
16	16 s		

Speedup and efficiency: an example

A program takes $T(1) = 120$ s on one processing unit. We run it on $N = 2, 4, 8, 16$ units and measure:

N	$T(N)$	$S(N) = T(1)/T(N)$	$E(N) = S(N)/N$
1	120 s	1.0	100%
2	62 s	1.9	97%
4	34 s	3.5	88%
8	21 s		
16	16 s		

Speedup and efficiency: an example

A program takes $T(1) = 120$ s on one processing unit. We run it on $N = 2, 4, 8, 16$ units and measure:

N	$T(N)$	$S(N) = T(1)/T(N)$	$E(N) = S(N)/N$
1	120 s	1.0	100%
2	62 s	1.9	97%
4	34 s	3.5	88%
8	21 s	5.7	71%
16	16 s		

Speedup and efficiency: an example

A program takes $T(1) = 120$ s on one processing unit. We run it on $N = 2, 4, 8, 16$ units and measure:

N	$T(N)$	$S(N) = T(1)/T(N)$	$E(N) = S(N)/N$
1	120 s	1.0	100%
2	62 s	1.9	97%
4	34 s	3.5	88%
8	21 s	5.7	71%
16	16 s	7.5	47%

Speedup and efficiency: an example

A program takes $T(1) = 120$ s on one processing unit. We run it on $N = 2, 4, 8, 16$ units and measure:

N	$T(N)$	$S(N) = T(1)/T(N)$	$E(N) = S(N)/N$
1	120 s	1.0	100%
2	62 s	1.9	97%
4	34 s	3.5	88%
8	21 s	5.7	71%
16	16 s	7.5	47%

- Speedup keeps growing, but efficiency drops: with 16 units, more than half of the machine is wasted.

Speedup and efficiency: an example

A program takes $T(1) = 120$ s on one processing unit. We run it on $N = 2, 4, 8, 16$ units and measure:

N	$T(N)$	$S(N) = T(1)/T(N)$	$E(N) = S(N)/N$
1	120 s	1.0	100%
2	62 s	1.9	97%
4	34 s	3.5	88%
8	21 s	5.7	71%
16	16 s	7.5	47%

- Speedup keeps growing, but efficiency drops: with 16 units, more than half of the machine is wasted.
- Where does the time go? Parts of the program that remain sequential, waiting for other units, communication, unequal work per unit, ...

Speedup and efficiency: an example

A program takes $T(1) = 120$ s on one processing unit. We run it on $N = 2, 4, 8, 16$ units and measure:

N	$T(N)$	$S(N) = T(1)/T(N)$	$E(N) = S(N)/N$
1	120 s	1.0	100%
2	62 s	1.9	97%
4	34 s	3.5	88%
8	21 s	5.7	71%
16	16 s	7.5	47%

- Speedup keeps growing, but efficiency drops: with 16 units, more than half of the machine is wasted.
- Where does the time go? Parts of the program that remain sequential, waiting for other units, communication, unequal work per unit, ...
- Is there a limit to the speedup? → Amdahl's law.

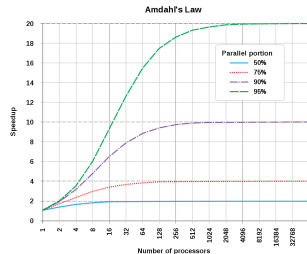
Amdahl's Law

How much faster can a program run? What is the limit?

- Gene Amdahl, 1967. Let s and p be the **sequential** and **parallel** fractions of a program ($s + p = 1$).
- Attainable speedup is limited by

$$S(N) = \frac{T(1)}{T(N)} \leq \frac{T(1)}{p T(1)/N + s T(1)} = \frac{1}{p/N + s} \leq \frac{1}{s}$$

- With $s = 10\%$, even infinitely many processors give at most $10\times$!
- Loss of hope? No: for most problems, s shrinks as the problem size grows (Gustafson's law).

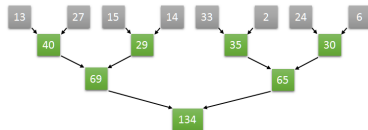


Amdahl's Law

Example: Sum of an array

$$\sum_{i=0}^{N-1} A[i]$$

- $N - 1$ additions in total
- For $N = 8$: 7 additions, of which 3 must be sequential, $S \leq 7/3 \approx 2.33$
- For $N = 16$: 15 additions, of which 4 must be sequential, $S \leq 15/4 = 3.75$
- In general: $N - 1$ additions, of which $\log_2 N$ must be sequential, $S \leq (N - 1)/\log_2 N$
- Note that at each step, all active units do **the same operation on different data**: this is the first parallel algorithm of the course, we will study it rigorously in the PRAM sessions.



Parallel sum

Example: Sum of an array with $N = 10^9$ elements

- Sequential: $N - 1 \approx 10^9$ additions, $T(1) \approx 10^9$ steps.

Example: Sum of an array with $N = 10^9$ elements

- Sequential: $N - 1 \approx 10^9$ additions, $T(1) \approx 10^9$ steps.
- **Intrinsically sequential part:** whatever the number of units, the result depends on all N inputs through a tree of additions of depth $\log_2 10^9 \approx 30$. These 30 steps cannot be parallelized.

Example: Sum of an array with $N = 10^9$ elements

- Sequential: $N - 1 \approx 10^9$ additions, $T(1) \approx 10^9$ steps.
- **Intrinsically sequential part**: whatever the number of units, the result depends on all N inputs through a tree of additions of depth $\log_2 10^9 \approx 30$. These 30 steps cannot be parallelized.
- **Parallel part**: the remaining $\approx 10^9$ additions are shared among the P units. Hence, in the spirit of Amdahl's law,

$$T(P) \approx \frac{10^9}{P} + 30, \quad S(P) = \frac{10^9}{10^9/P + 30}.$$

Example: Sum of an array with $N = 10^9$ elements

- Sequential: $N - 1 \approx 10^9$ additions, $T(1) \approx 10^9$ steps.
- **Intrinsically sequential part**: whatever the number of units, the result depends on all N inputs through a tree of additions of depth $\log_2 10^9 \approx 30$. These 30 steps cannot be parallelized.
- **Parallel part**: the remaining $\approx 10^9$ additions are shared among the P units. Hence, in the spirit of Amdahl's law,

$$T(P) \approx \frac{10^9}{P} + 30, \quad S(P) = \frac{10^9}{10^9/P + 30}.$$

- $P = 1000$: $T \approx 10^6 + 30$, $S \approx 1000$, $E \approx 100\%$.

Example: Sum of an array with $N = 10^9$ elements

- Sequential: $N - 1 \approx 10^9$ additions, $T(1) \approx 10^9$ steps.
- **Intrinsically sequential part**: whatever the number of units, the result depends on all N inputs through a tree of additions of depth $\log_2 10^9 \approx 30$. These 30 steps cannot be parallelized.
- **Parallel part**: the remaining $\approx 10^9$ additions are shared among the P units. Hence, in the spirit of Amdahl's law,

$$T(P) \approx \frac{10^9}{P} + 30, \quad S(P) = \frac{10^9}{10^9/P + 30}.$$

- $P = 1000$: $T \approx 10^6 + 30$, $S \approx 1000$, $E \approx 100\%$.
- $P = 10^6$: $T \approx 10^3 + 30$, $S \approx 9.7 \times 10^5$, $E \approx 97\%$.

Example: Sum of an array with $N = 10^9$ elements

- Sequential: $N - 1 \approx 10^9$ additions, $T(1) \approx 10^9$ steps.
- **Intrinsically sequential part**: whatever the number of units, the result depends on all N inputs through a tree of additions of depth $\log_2 10^9 \approx 30$. These 30 steps cannot be parallelized.
- **Parallel part**: the remaining $\approx 10^9$ additions are shared among the P units. Hence, in the spirit of Amdahl's law,

$$T(P) \approx \frac{10^9}{P} + 30, \quad S(P) = \frac{10^9}{10^9/P + 30}.$$

- $P = 1000$: $T \approx 10^6 + 30$, $S \approx 1000$, $E \approx 100\%$.
- $P = 10^6$: $T \approx 10^3 + 30$, $S \approx 9.7 \times 10^5$, $E \approx 97\%$.
- $P \rightarrow \infty$: $T \rightarrow 30$, $S \rightarrow 10^9/30 \approx 3 \times 10^7$: the 30 sequential steps are the bottleneck (Amdahl's $1/s$ bound).

Example: Sum of an array with $N = 10^9$ elements

- Sequential: $N - 1 \approx 10^9$ additions, $T(1) \approx 10^9$ steps.
- **Intrinsically sequential part**: whatever the number of units, the result depends on all N inputs through a tree of additions of depth $\log_2 10^9 \approx 30$. These 30 steps cannot be parallelized.
- **Parallel part**: the remaining $\approx 10^9$ additions are shared among the P units. Hence, in the spirit of Amdahl's law,

$$T(P) \approx \frac{10^9}{P} + 30, \quad S(P) = \frac{10^9}{10^9/P + 30}.$$

- $P = 1000$: $T \approx 10^6 + 30$, $S \approx 1000$, $E \approx 100\%$.
- $P = 10^6$: $T \approx 10^3 + 30$, $S \approx 9.7 \times 10^5$, $E \approx 97\%$.
- $P \rightarrow \infty$: $T \rightarrow 30$, $S \rightarrow 10^9/30 \approx 3 \times 10^7$: the 30 sequential steps are the bottleneck (Amdahl's 1/s bound).
- As long as $N/P \gg 30$, the sequential part is negligible: it shrinks relative to the problem size (Gustafson).

- 1 Introduction
- 2 Parallel computing, why?
- 3 Measuring parallel performance
- 4 Classification of parallel machines**
- 5 Kinds of parallelism in programs
- 6 Parallel programming models
- 7 Course organisation

Flynn's classification of parallel machines

A 2×2 matrix to classify parallel machines

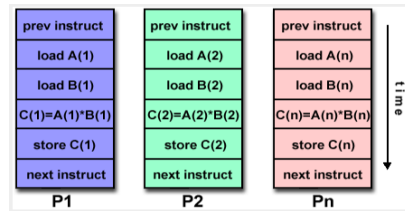
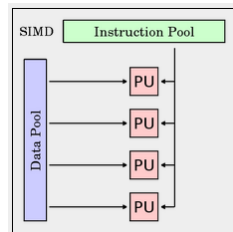
- Popular classification, used since 1966
- Rows: single/multiple **instruction** streams
- Columns: single/multiple **data** streams
- **SISD**: the sequential processor of the previous slides
- **SIMD** and **MIMD**: the two families of parallel machines we program in this course
- **MISD**: rare, not covered

SISD Single Instruction stream Single Data stream	SIMD Single Instruction stream Multiple Data stream
MISD Multiple Instruction stream Single Data stream	MIMD Multiple Instruction stream Multiple Data stream

Flynn's classification

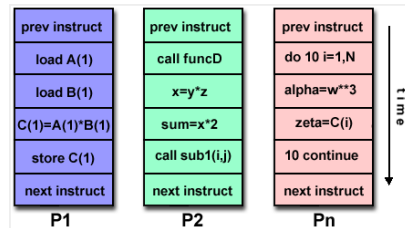
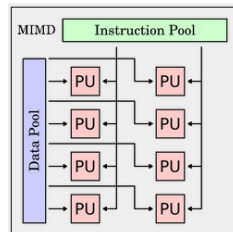
Single Instruction, Multiple Data (SIMD)

- “Single” instruction: all compute units execute **the same instruction** at each clock cycle
- “Multiple” data: each compute unit operates on **a different data element**
- Execution is synchronous (in lockstep) and deterministic
- Useful for numerous problems and applications (matrix computations, image processing, ...)
- **This is what we will learn in today's session:** the vector units of a CPU (AVX) execute one instruction on 8 floats at once.



Multiple Instruction, Multiple Data (MIMD)

- “Multiple” instructions: each compute unit executes **its own instruction stream**
- “Multiple” data: each compute unit operates on **a different data element**
- Execution can be asynchronous and non-deterministic
- Most parallel machines today: multi-core CPUs, GPUs, clusters
- **This is what we program in the next sessions:** OpenMP threads, CUDA threads, MPI processes.



- 1 Introduction
- 2 Parallel computing, why?
- 3 Measuring parallel performance
- 4 Classification of parallel machines
- 5 Kinds of parallelism in programs**
- 6 Parallel programming models
- 7 Course organisation

From machines to programs

Flynn's classification tells us **what the hardware can do**: execute one instruction stream on many data (SIMD), or many instruction streams (MIMD).

It says nothing about **where the parallelism comes from in a program**. Looking at programs and algorithms instead, we find two kinds:

- **Data parallelism**: the *same* operation must be applied to many data elements.
- **Task parallelism**: *different* pieces of work can proceed independently.

Both can run on SIMD or MIMD hardware; data parallelism is the natural fit for SIMD, and the one we will meet most often in this course.

Data parallelism

Carry out the same operation on every element of a vector/array.

- $C[i] = A[i] + B[i]$ for all i : **N independent operations**
- Maps naturally to SIMD hardware, but also to threads and processes (each one takes a slice of i)
- Most processors have efficient hardware for such computations (vector units in CPUs and GPUs)
- Very widespread in scientific computing, AI, computer graphics, signal processing, ...

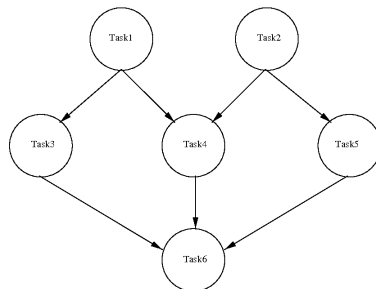
index	0	1	2	3	4	5	6	7
Array A	1	2	3	1	4	1	6	7
					+			
B	1	2	3	1	5	2	6	1
					=			
C	2	4	6	2	9	3	12	8

Sum of two arrays

Task parallelism

Execute a set of tasks in parallel while respecting their dependencies.

- Examples of independent tasks?
 - Function calls: $\mathbf{a} = \mathbf{f}(\mathbf{x}); \mathbf{b} = \mathbf{g}(\mathbf{y});$
 - Independent blocks of code in a function/program
 - Execution of a single program with different parameters (e.g., hyperparameter search)
- Each task executes on a separate compute unit.
- Dependencies must be respected if they exist.
 - Load balancing might be a challenge.
 - Effective scheduling of tasks is an entire research domain in itself.
- **Not covered in this course:** task parallelism is the subject of next trimester's *High-Performance Computing on Multicore Architectures* course.



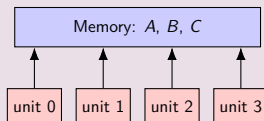
Task parallelism

- 1 Introduction
- 2 Parallel computing, why?
- 3 Measuring parallel performance
- 4 Classification of parallel machines
- 5 Kinds of parallelism in programs
- 6 Parallel programming models**
- 7 Course organisation

Two memory models

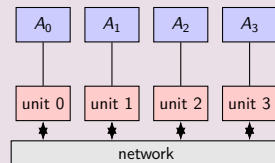
Where does the data live, and who can see it?

Shared memory



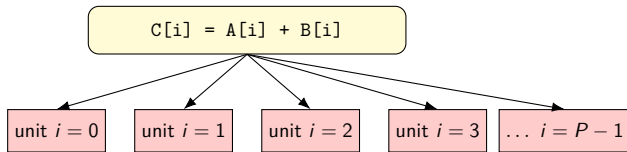
- All units see the **same arrays**; unit i reads and writes $A[i]$ directly.
- SIMD lanes, OpenMP threads, CUDA threads.

Distributed memory



- Each unit owns **its own piece** of the data; exchange requires **explicit messages**.
- MPI processes.

Same program, different index



All processing units execute the same program; only the value of i differs.

Paradigm	Processing unit	Index i
SIMD (today)	one lane of a vector register	position in the vector (0..7)
OpenMP	thread on a CPU core	thread id
CUDA	thread on a GPU	thread index
MPI	process on a machine of a cluster	rank
PRAM (algorithm sessions)	abstract processor	processor number

PRAM algorithms are written exactly in this model; the four programming sessions are four ways of running them on real hardware.

- 1 Introduction
- 2 Parallel computing, why?
- 3 Measuring parallel performance
- 4 Classification of parallel machines
- 5 Kinds of parallelism in programs
- 6 Parallel programming models
- 7 Course organisation**

Roadmap of the course

Sessions, in order

- 1 **SIMD** programming with AVX (today)
- 2 Parallel algorithms: **PRAM** model, part 1
- 3 Multi-core programming with **OpenMP**
- 4 Parallel algorithms: **PRAM** model, part 2
- 5 GPU programming with **CUDA**
- 6 Parallel algorithms on **communication networks**
- 7 Distributed memory programming with **MPI**

Follow-up courses

- **High-Performance Computing on Multicore Architectures**
- **MPI Programming on Distributed Memory Architectures**
- **GPU Programming**

Not in this course

How the hardware is built in depth (caches, NUMA, interconnects, GPU internals) and how to squeeze the last percent of performance out of it.

Contact

Oguz Kaya

Université Paris-Saclay and LISN, Paris, France

oguz.kaya@universite-paris-saclay.fr

www.oguzkaya.com